END-TO-END DATA ENGINEERING + ML PROJECT

Retail Analytics
ETL + ML Forecasting

A production-grade pipeline covering data ingestion, transformation, PostgreSQL storage, SQL analytics, three ML forecasting models, MLflow experiment tracking, and a live Streamlit dashboard.

TOTAL REVENUE	TOTAL CUSTOMERS	TOTAL ORDERS	ML MODELS BUILT
£7.57M	4,277	15,195	3

PIPELINE FLOWCHART

STEP 1 Extract → STEP 2 Transform → STEP 3 Load → STEP 4 Analyze

STEP 5 ML Models → STEP 6 Dashboard → STEP 7 Deploy

Data flow: Raw CSV → ETL Pipeline → PostgreSQL → SQL Analysis → 3 ML Models (Linear Regression + Prophet + LSTM) → MLflow Tracking → Streamlit Dashboard → FastAPI + Docker

STEP-BY-STEP PIPELINE WALKTHROUGH

1 EXTRACT — Load Raw Retail Data from CSV

The process begins in `etl_pipeline.py`. A raw transactional CSV is loaded into a Pandas DataFrame using `pd.read_csv()`. The dataset contains retail invoice data — product codes, quantities, prices, customer IDs, invoice dates, and country fields.

- Source: `cleaned_data.csv` (UK retail transaction dataset)
- Columns confirmed: Invoice, StockCode, Description, Quantity, InvoiceDate, Price, Customer ID, Country
- Initial column validation is printed to confirm schema integrity before processing

2 TRANSFORM — Clean, Validate & Engineer Features

Data quality is enforced by removing null Customer IDs, filtering out returns (negative quantities) and invalid prices, parsing invoice dates, and computing a derived `TotalPrice = Quantity × Price` column.

- Drop rows where `Customer ID` is null (data quality)
- Filter: `Quantity > 0` and `Price > 0` (removes cancellations)
- Parse `InvoiceDate` to datetime with error coercion
- Feature engineering: `TotalPrice` as a derived monetary metric

3 LOAD — Persist to PostgreSQL via psycopg2

Cleaned data is inserted row-by-row into a PostgreSQL table (`retail_data`) using `psycopg2`. This creates a persistent, queryable data store that acts as the single source of truth for all downstream analytics and ML.

- Connection managed with environment-driven config (host, db, user, password)
- Row-level parameterised INSERT prevents SQL injection
- Commit on success, connection teardown on completion
- The Streamlit app then reads from this DB via `pd.read_sql()`

4 ANALYZE — SQL-Based Aggregations & Business Metrics

With data in PostgreSQL, SQL queries power all KPI aggregations: total revenue, unique customers, order counts, revenue by country, top products by volume, and top customers by spend. Pandas groupby operations extend this analysis within Python.

- KPIs: SUM(totalprice), COUNT(DISTINCT customerid), COUNT(DISTINCT invoice)
- Segmentation: revenue grouped by country, product, and customer
- Time series: daily sales by invoicedate for trend visibility

MACHINE LEARNING

Three Forecasting Models + Churn Prediction

5

Three sales forecasting approaches of increasing sophistication were implemented for comparison, plus a churn prediction model using RFM (Recency-Frequency-Monetary) customer segmentation.

- **Linear Regression** (baseline): day-number as a numeric feature → predicts next 30 days of revenue
- **Prophet** (intermediate): Facebook's time-series model with seasonality, trend decomposition, and confidence intervals
- **LSTM** (advanced): deep learning sequence model capturing long-range temporal patterns
- **Churn Model**: logistic regression on RFM features — customers with Recency > 90 days labelled as churned
- All experiments tracked with **MLflow**: parameters, metrics, and artifacts logged per run

DASHBOARD

Interactive Streamlit Analytics Dashboard

6

A Streamlit app provides a live, interactive view of all analytics and ML outputs. Users can filter by country, view KPI metrics, explore product and customer rankings, and inspect all three forecast models side-by-side.

- Country filter sidebar dynamically slices all charts and KPIs
- Bar charts for top products, top customers, and revenue by country
- Line chart for daily sales trend
- Forecast tables (CSV-backed) and forecast charts (PNG images) for all three models

DEPLOYMENT

FastAPI Serving + Docker Containerization

7

The pipeline is wrapped for production deployment using FastAPI as a model-serving REST layer and Docker for container-based environment consistency — enabling the solution to run identically in development, staging, and production environments.

- FastAPI exposes trained models as REST endpoints for real-time inference
- Docker encapsulates all dependencies (Python, PostgreSQL, Streamlit, ML libs) into a reproducible container image
- MLflow serves as the experiment registry — enabling model versioning and rollback

FILE BREAKDOWN

etl_pipeline.py

The backbone of the project. Implements the full Extract-Transform-Load workflow: loads the raw CSV, enforces data quality rules, engineers the TotalPrice feature, and inserts clean records into PostgreSQL row by row via psycopg2.

ETL · PostgreSQL · pandas

app.py

The Streamlit dashboard application. Reads live data from PostgreSQL, renders KPI cards, bar and line charts, and displays all three forecast outputs (tables + PNG images). Includes a country-based sidebar filter that dynamically slices all visuals.

Streamlit · Dashboard · SQL

forecast_model.py

Implements the baseline Linear Regression forecaster. Aggregates daily sales, encodes dates as numeric day-numbers, trains a sklearn LinearRegression model, predicts 30 days forward, saves the forecast CSV and chart, and logs everything to MLflow as a tracked experiment run.

Linear Regression · MLflow · sklearn

churn_model.py

Customer churn prediction using RFM analysis. Computes Recency (days since last purchase), Frequency (unique invoice count), and Monetary (total spend) per customer. Labels customers as churned if Recency > 90 days, then trains a Logistic Regression classifier and prints accuracy and classification report.

RFM Analysis · Churn · LogisticRegression

KEY CODE SNIPPETS

1 — ETL: Data Cleaning & Feature Engineering

PYTHON · ETL_PIPELINE.PY — clean_dataframe()

```python
def clean_dataframe(df):
    # Remove rows with missing customer identifiers
    df = df.dropna(subset=["Customer ID"])

    # Filter out returns (negative qty) and free items (price = 0)
    df = df[(df["Quantity"] > 0) & (df["Price"] > 0)]

    # Parse dates, drop rows that failed to parse
    df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"], errors="coerce")
    df = df.dropna(subset=["InvoiceDate"])
```

```python
    # Feature engineering: derive revenue per line item
    df["TotalPrice"] = df["Quantity"] * df["Price"]

    return df
```

2 — RFM Churn Feature Engineering

PYTHON · CHURN_MODEL.PY — RFM customer segmentation

```python
# --- Recency: days since last purchase ---
latest_date = df["InvoiceDate"].max()
recency = df.groupby("Customer ID")["InvoiceDate"].max().reset_index()
recency["Recency"] = (latest_date - recency["InvoiceDate"]).dt.days

# --- Frequency: unique invoices per customer ---
frequency = df.groupby("Customer ID")["Invoice"].nunique().reset_index()
frequency.columns = ["Customer ID", "Frequency"]

# --- Monetary: total revenue per customer ---
monetary = df.groupby("Customer ID")["TotalPrice"].sum().reset_index()

# Merge into RFM table and create binary churn label
rfm = recency.merge(frequency, on="Customer ID").merge(monetary, on="Customer ID")
rfm["Churn"] = rfm["Recency"].apply(lambda x: 1 if x > 90 else 0)
```

3 — SQL: Business KPI Queries (run via psycopg2 / Streamlit)

SQL — Key analytical queries

```sql
-- KPI: Total revenue, customer count, order count
SELECT
    SUM(totalprice)            AS total_revenue,
    COUNT(DISTINCT customerid) AS unique_customers,
    COUNT(DISTINCT invoice)    AS total_orders
FROM retail_data;

-- Top 10 products by quantity sold
SELECT description, SUM(quantity) AS total_qty
FROM retail_data
GROUP BY description
ORDER BY total_qty DESC
LIMIT 10;

-- Revenue by country
```

```sql
SELECT country, SUM(totalprice) AS revenue
FROM retail_data
GROUP BY country
ORDER BY revenue DESC
LIMIT 10;
```

TOOLS USED AT EACH STAGE

STAGE	TOOLS / LIBRARIES	PURPOSE
Extract	`pandas` `CSV`	Load raw retail transactions from flat file
Transform	`pandas` `numpy`	Null removal, type casting, feature engineering
Load	`PostgreSQL` `psycopg2`	Persist clean data in a relational database
Analysis	`SQL` `pandas groupby`	KPIs, segmentations, aggregations
ML – Baseline	`scikit-learn` `LinearRegression`	Simple numeric-day trend forecasting
ML – Intermediate	`Prophet` `matplotlib`	Seasonality-aware time-series forecasting
ML – Advanced	`TensorFlow / Keras` `LSTM`	Deep learning sequence model for sales prediction
Churn	`scikit-learn` `LogisticRegression`	RFM-based binary churn classification
Experiment Tracking	`MLflow`	Log params, metrics, artifacts per model run
Dashboard	`Streamlit` `pandas`	Live interactive analytics UI
Model Serving	`FastAPI`	REST API endpoint for real-time model inference
Containerization	`Docker`	Reproducible environment packaging

WHAT TO SHOW IN A REVIEW

SLIDE 01

Dashboard Hero — KPI Cards + Filter

Show the £7.57M revenue, 4,277 customers, 15,195 orders KPI row with the country sidebar filter active. This instantly communicates business scope and real data at scale.

SLIDE 02

Top Products & Top Customers Charts

The bar charts for top 10 products (PAPER CRAFT dominates) and top customers by revenue demonstrate SQL aggregation skill and Streamlit charting fluency.

SLIDE 03

Sales Trend Line Chart (2009–2011)

The multi-year daily sales trend line shows you understand time-series data, seasonal patterns, and can process date columns correctly at scale.

SLIDE 04

Prophet Forecast Chart + Table

Show the Prophet chart with confidence interval bands (yhat_lower / yhat_upper). Explains that you understand uncertainty quantification — a key production ML concept.

SLIDE 05

3-Model Comparison (LR vs Prophet vs LSTM)

Showing all three forecasts side-by-side is a power move — it demonstrates model selection thinking, not just model building. Mention trade-offs: LR = fast/interpretable, Prophet = seasonality-aware, LSTM = pattern depth.

SLIDE 06

MLflow Experiment Tracking UI

A screenshot of the MLflow UI at localhost:5000 showing logged runs, parameters (model name, forecast horizon), and artifacts is proof of production engineering mindset — not just notebook coding.

SLIDE 07

ETL Pipeline Terminal Output

"Step 1: Loading CSV... Step 2: Cleaning data... Step 3: Inserting X rows..." — shows the pipeline runs end-to-end cleanly and you understand process logging.

SLIDE 08

Churn Model Classification Report

The scikit-learn classification report (precision, recall, F1, support) shows you evaluate models properly, not just train them. Mention the 90-day RFM threshold logic as a business-driven design decision.

BUSINESS IMPACT STATEMENT

"This project delivers a complete, data-driven retail intelligence system — from raw transactions to

actionable forecasts — replacing manual reporting with an automated, always-live pipeline."

By automating ETL, centralising data in PostgreSQL, and surfacing KPIs through an interactive dashboard, this system enables retail managers to make faster, evidence-based decisions on inventory, promotions, and customer retention. The three-model forecasting suite provides a 30-day revenue outlook, while the churn model proactively flags at-risk customers — enabling targeted re-engagement before revenue is lost.

- 📉 Reduced manual reporting effort
- 📈 30-day revenue forecasting
- 🔁 Automated ETL pipeline
- 🎯 Proactive churn identification
- 🔬 Reproducible ML experiments via MLflow
- 📦 Containerised for any environment

INTERVIEW PITCHES

30-Second Answer | 2-Minute Answer

"I built a **full end-to-end retail analytics and ML pipeline**. It starts with a Python ETL script that extracts raw retail transaction data, cleans it, and loads it into a PostgreSQL database. From there, I ran SQL analysis to compute business KPIs like total revenue, top customers, and country-level sales. I then built **three forecasting models** — a Linear Regression baseline, Facebook Prophet for seasonality, and an LSTM deep learning model — all tracked using MLflow. Everything is surfaced through a **live Streamlit dashboard** with filters and charts. The pipeline is served via FastAPI and containerised with Docker, making it production-ready."